



KARAN

The app itself: users, data, flows and the demo

You open the presentation and explain what the app is, who uses it, how data moves, and you run the walkthrough.

1. Your role in one sentence

You explain the application as a whole, and you own the code that turns each screen into an action: the student and officer flows, the dashboards, and the demo data.

2. What you own

FILE	LINES	WHAT IT IS
<code>ejb/DataSeeder.java</code>	336	Singleton bean that fills an empty database: 3 officers, 25 students, 10 companies, 14 drives, 85 applications
<code>web/StudentDashboardServlet.java</code>	88	The student's landing page
<code>web/DriveBrowseServlet.java</code>	90	The drives board and a single drive, with the eligibility verdict for each
<code>web/ApplicationServlet.java</code>	127	Apply, withdraw, and the "my applications" list
<code>web/AdminDashboardServlet.java</code>	66	The officer's landing page and its totals
<code>web/AdminDriveServlet.java</code>	220	Announce, edit, open, close and delete drives, with validation
<code>web/AdminCompanyServlet.java</code>	126	The recruiter register
<code>web/AdminStudentServlet.java</code>	108	Student register, a student's record, suspend and unlock
<code>web/AdminApplicationServlet.java</code>	107	The pipeline: filter, and move a candidate's status with a note
<code>web/ShortlistServlet.java</code>	159	Batch shortlist using the stateful bean
<code>web/AuditServlet.java</code> , <code>web/SystemServlet.java</code>	100	The audit trail and the system page
<code>run.ps1</code> , <code>README.md</code> , <code>docs/team/</code>		How the app is started, reseeded and explained

3. The two minute explanation

SAY THIS

What it is. Every placement season a college has to match students to company drives. This app is that matching desk. It has two kinds of user: students and the placement officer.

Student side. A student registers with a real email and their academic record: branch, CGPA, backlogs. From then on the app compares that record with the rules on every drive. Drives they qualify for show an Apply button. Drives they do not qualify for show the exact reason, for example "minimum CGPA is 7.5, yours is 6.4". They apply, and then follow the status and the officer's notes.

Officer side. The officer maintains the companies, announces drives with their rules, moves candidates from applied to shortlisted to interview to selected, and can shortlist many at once. There are reports that come straight from the database with plain JDBC, a page that shows the database itself, and an audit trail of everything that changed.

Behind it. Servlets receive requests, EJB beans hold the business rules, JPA and Hibernate save the objects to MySQL, and JSP pages draw the screens. The reports use JDBC directly on the same connection pool. Each of us will now explain our own layer.

4. The story, step by step

A student's journey

- 1 **Register** at `/register`. Name, college email, password, roll number, branch, year, CGPA, backlogs, phone. The row goes to `app_user` and `student_profile`. The sign in page then shows a "Welcome, your account is ready" panel with the email filled in. (Tiya explains how.)
- 2 **Sign in** at `/login`. The servlet checks the password hash and stores a small `SessionUser` in the HttpSession.
- 3 **Dashboard** at `/app/dashboard`: profile summary, counts of your applications by status, how many open drives you can still apply to, and, for a new student, a getting started checklist (create account, finish profile, apply to a first drive) with a progress bar and the next button to press.
- 4 **Drives board** at `/app/drives`. `DriveBrowseServlet` asks `DriveService.check` for each open drive and shows the verdict.
- 5 **Apply**: open a drive, press Apply. `ApplicationServlet.doPost` checks the rules again, then `ApplicationService.apply` writes the row and queues a notification message.
- 6 **Follow it** at `/app/applications`: status and the cell's latest note. A student may withdraw.

The officer's journey

- 1 **Sign in** with an officer account. There is no public way to become an officer.

2 Overview at `/admin/dashboard` : how many students, how many placed, open drives, applications.

3 Companies and drives: add a recruiter, announce a drive with role, package, location, minimum CGPA, backlog limit, branches and dates. Every field is validated on the server.

4 Pipeline at `/admin/applications` : filter by drive and status, move a candidate on, write a note the student will see.

5 Shortlist at `/admin/shortlist` : tick many candidates, then commit them together. A stateful bean holds the selection between clicks.

6 Reports, Database, Audit: Yuvraj and Chandra explain these.

5. The demo data, and why it looks complete

`DataSeeder` is a singleton bean marked `@Startup` . On deploy it counts the accounts. If there are none, it fills the database. If there are some, it does nothing, so restarting never wipes real work.

TABLE	ROWS	WHAT IS IN IT
<code>app_user</code>	28	3 officers and 25 students. Every password is stored as its own salted PBKDF2 hash.
<code>student_profile</code>	25	Branches: 8 Computer Science, 6 Information Technology, 4 Electronics, 2 Statistics, 2 Mechanical, 2 Civil (one Civil student is suspended). CGPA runs from 5.6 to 9.3. Three have no phone number, so the profile warning appears.
<code>company</code>	10	Northwind Systems, Blue Harbour Analytics, Vertex Financial, Lumen Health, Kestrel Logistics, Orbit Semiconductors, Meridian Consulting, Paperkite Learning, Ironbridge Infrastructure, Saffron Retail Tech
<code>drive</code>	14	10 open with deadlines from 3 to 24 days away, and 4 already closed, so the archive has history. Rules differ: package 5.0 to 12.5 LPA, minimum CGPA 5.5 to 8.0, backlogs 0 to 3, different branch lists.
<code>job_application</code>	85	Every status appears: applied, shortlisted, interview, selected, rejected. 6 students hold an offer.

How the applications were made, in case you are asked

Six are written by hand to give the demo accounts a story. The rest come from simple arithmetic over the student and drive lists, and only for pairs where the student really meets the drive's rules, using the same three tests as the live app. Nothing is random, so the data is the same every time and the demo can be rehearsed. Every date is counted from today.

CAREFUL

Because dates are counted from the day the data was created, a database seeded last week has drives whose deadline has passed, and students cannot apply. Before the presentation run `.\run.ps1 -Reseed` . It empties the tables and lets the seeder fill them again with fresh dates.

6. Code walk in VS Code

- 1 **Open** `DriveBrowseServlet.java` . Point at the loop that calls `driveService.check(profile, drive)` for every drive and puts the map of verdicts on the request. Then `Web.render(request, response, "drives.jsp", "drives")` . Say: the servlet decides nothing, it asks the bean and hands the answer to a page.
- 2 **Open** `ApplicationServlet.java` , `doPost` . Show the order: find the drive, `check` the verdict, refuse if already applied or not eligible, then `applicationService.apply` , then a flash message and a redirect. Say: the button was hidden already, but the rules run again here because a form can be replayed or written by hand.
- 3 **Same file, `Web.redirect` after the write.** Say: after a POST we redirect instead of forwarding, so pressing refresh cannot apply twice (Post, Redirect, Get).
- 4 **Open** `AdminDriveServlet.java` . Show the block of `errors.put(...)` lines: role, package range, CGPA range, at least one branch, last date on or before the drive date.
- 5 **Open** `ShortlistServlet.java` . Show `basket()` : a JNDI lookup of `java:module/ShortlistBean` stored in the HttpSession. Say why it is not injected: a servlet is one shared object, a stateful bean must belong to one officer.
- 6 **Open** `DataSeeder.java` . Show `@Singleton @Startup` , the "count accounts" guard at the top of `seed()` , and the `meets` method used to make only valid applications.

7. Questions you will be asked

Q Who exactly uses this app?

Two kinds of people. Students, who register themselves, and placement officers, who are created by the college. Companies do not log in: the officer keeps their details and posts their drives, as a real placement cell does.

Q How does data get into the system?

Through forms. A student registration creates the account and profile. A student applying creates an application. An officer creating a company or a drive creates those rows. An officer can also import many students at once. Everything passes server side validation before it reaches the database, and the demo data is loaded once by the seeder.

Q How does data get out?

Students see a board of drives with their eligibility and their own applications. Officers see totals, a pipeline, reports read with JDBC, a browser for the database itself, and an audit trail. A student can also download their own resume.

Q Why can an officer not register on the public page?

Because the register form only ever creates a student. An officer account is a privilege, so it is created by the college. Only the seeder creates officers here. That keeps a student from choosing their own role.

Q Why are there no recruiter logins?

It was a scope decision. The cell is the go-between, so the officer posts drives for the company. A recruiter role would be a natural extension: a third role, its own pages, and an approval step by the officer.

Q A student says a drive is missing. Why?

The default board shows open drives. The board also has an "all" view. A drive whose last date has passed shows as not eligible with the reason "the last date to apply was ...", so nothing is hidden without explanation.

Q Is the eligibility check only on the page?

No. The page hides the button, and the servlet checks the rules again on the POST, because a page can be stale and a request can be hand written.

Q Where does the demo data come from, and is it real?

It is made up. The seeder creates it on the first start if the database is empty. It is deterministic, so it is the same every time, and the dates are counted from today so the drives are always current.

Q What did you not build?

Recruiter logins, email or SMS delivery of the notifications (a message is queued through JMS and handled by a bean, not sent by email), password reset by email, and confirmation links for new accounts. Two smaller gaps: nothing stops a student who already holds an offer from applying to more drives (the dashboard says the cell will be in touch about the policy), and the audit trail records the actor as "portal" instead of the individual officer, because sign in is done by the app's own filter rather than a server security realm. These are the honest next steps.

Q Which technologies are used and why?

Servlets and JSP for the web layer, EJB for business rules and transactions, JPA with Hibernate for ordinary reads and writes, JDBC for reports where SQL fits better than objects, MySQL as the database, GlassFish as the Jakarta EE server. Each one is a topic from the course.

8. If something goes wrong

SYMPTOM	FIX
Page will not load	Is MySQL running? Is the server up? Run <code>.\run.ps1</code> and wait for "deployed".

Drives all say "the last date has passed"	Stale seed dates. Run <code>.\run.ps1 -Reseed</code> .
"That form could not be verified"	The page sat open too long or the server restarted. Reload the page and try again.
Login fails for everyone	The database may be empty or half filled. <code>.\run.ps1 -Reseed</code> .